Resumo e Exemplos Práticos em SQLite

Junções de dados, funções de agregação e subconsultas

Material didático de Banco de Dados

Sumário

1 Objetivos da aula **2**

2 Preparação do banco de dados **2**

3 Junção horizontal e vertical de dados **2**

3.1 Junção horizontal: `JOIN` 2

3.2 Junção vertical: `UNION` 3

4 Funções de agregação em bancos de dados **3**

4.1 Agrupamento com `GROUP BY` 4

4.2 Filtro de grupos com `HAVING` 4

5 Subconsultas em bancos de dados **4**

6 Comparação dos assuntos **5**

7 Atividades propostas **5**

A Script SQL integral para SQLite Online **5**

1 Objetivos da aula

Compreender a combinação de dados provenientes de tabelas, o cálculo de estatísticas por meio de funções de agregação e a utilização de consultas SQL aninhadas. Os exemplos podem ser executados no SQLite Online.

2 Preparação do banco de dados

Será utilizada uma pequena base de uma loja de dispositivos eletrônicos, composta pelas tabelas `clientes`, `produtos`, `pedidos` e `itens_pedido`. Há ainda duas tabelas auxiliares para demonstrar `UNION`.

Procedimento: selecione o mecanismo SQLite no site, execute primeiro os comandos de criação e inserção da Seção A e depois teste as consultas em separado. O script foi organizado para poder ser executado integralmente desde o início em um banco de teste.

3 Junção horizontal e vertical de dados

3.1 Junção horizontal: `JOIN`

A junção horizontal reúne **colunas** de tabelas diferentes a partir de condições de relacionamento. No caso de clientes e pedidos, a coluna `id_cliente` estabelece a correspondência entre as tabelas.

Operador	Finalidade
`INNER JOIN`	Retorna somente linhas com correspondência entre as tabelas.
`LEFT JOIN`	Inclui todas as linhas da tabela à esquerda, preenchendo com `NULL` quando não há correspondência.
`CROSS JOIN`	Produz todas as combinações possíveis entre linhas das tabelas.

Exemplo 1: clientes e respectivos pedidos.

```sql
SELECT c.nome, p.id_pedido, p.data_pedido
FROM clientes AS c
INNER JOIN pedidos AS p
    ON c.id_cliente = p.id_cliente;
```

A cliente Ana aparece duas vezes porque possui dois pedidos. A cliente Elisa não aparece porque não realizou pedidos.

Exemplo 2: todos os clientes, inclusive sem pedidos.

```sql
SELECT c.nome, p.id_pedido
FROM clientes AS c
LEFT JOIN pedidos AS p
    ON c.id_cliente = p.id_cliente;
```

Neste caso, Elisa aparece com `NULL` no identificador do pedido.

3.2 Junção vertical: `UNION`

A junção vertical reúne **linhas** produzidas por consultas distintas. As consultas precisam ter o mesmo número de colunas e tipos de dados compatíveis.

Operador	Finalidade
`UNION`	Combina os resultados, eliminando linhas repetidas.
`UNION ALL`	Combina os resultados, mantendo repetições.

Exemplo 3: clientes cadastrados em anos diferentes.

```
SELECT nome, cidade FROM clientes_2025
UNION
SELECT nome, cidade FROM clientes_2026;
```

Exemplo 4: preservar registros repetidos.

```
SELECT nome, cidade FROM clientes_2025
UNION ALL
SELECT nome, cidade FROM clientes_2026;
```

O par `('Ana','Sao Luis')` aparece uma vez com `UNION` e duas vezes com `UNION ALL`.

4 Funções de agregação em bancos de dados

As funções de agregação resumem conjuntos de registros e facilitam relatórios estatísticos.

Função	Descrição
`COUNT(*)`	Conta todas as linhas do resultado.
`COUNT(coluna)`	Conta os valores não nulos da coluna.
`SUM()`	Soma valores numéricos.
`AVG()`	Calcula a média dos valores não nulos.
`MIN()`	Obtém o menor valor.
`MAX()`	Obtém o maior valor.

Exemplo 5: contar clientes.

```
SELECT COUNT(*) AS total_clientes FROM clientes;
```

Resultado: **5 clientes**.

Exemplo 6: estatísticas de preço.

```
SELECT ROUND(AVG(preco), 2) AS preco_medio,
       MIN(preco) AS menor_preco,
       MAX(preco) AS maior_preco
FROM produtos;
```

Com os registros da base: preço médio de **R$ 45,60**, mínimo de **R$ 18,00** e máximo de **R$ 90,00**.

4.1 Agrupamento com `GROUP BY`

O comando `GROUP BY` forma grupos de linhas com valores em comum, permitindo calcular estatísticas para cada grupo.

```
SELECT categoria, COUNT(*) AS quantidade
FROM produtos
GROUP BY categoria;
```

As categorias `Sensores` e `Placas` possuem dois produtos cada; `Modulos` possui um.

4.2 Filtro de grupos com `HAVING`

O `WHERE` filtra linhas **antes** do agrupamento. O `HAVING` filtra grupos **depois** de calculados os valores agregados.

```
SELECT ip.id_pedido,
       SUM(ip.quantidade * pr.preco) AS total
FROM itens_pedido AS ip
JOIN produtos AS pr ON pr.id_produto = ip.id_produto
GROUP BY ip.id_pedido
HAVING SUM(ip.quantidade * pr.preco) > 100;
```

Nesse conjunto de dados, o pedido 101 totaliza R$ 115,00 e o pedido 104 totaliza R$ 130,00. Ambos satisfazem o filtro.

5 Subconsultas em bancos de dados

Uma subconsulta é um `SELECT` utilizado dentro de outra instrução SQL. Pode retornar um valor único, uma coluna de valores ou um conjunto de linhas, conforme seu uso.

Exemplo 7: produtos mais caros que a média geral.

```
SELECT descricao, preco
FROM produtos
WHERE preco > (
    SELECT AVG(preco) FROM produtos
);
```

A subconsulta retorna o valor médio de R$ 45,60. A consulta externa apresenta `ESP32` e `Arduino Uno`.

Exemplo 8: clientes que realizaram pedidos.

```
SELECT nome
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente FROM pedidos
);
```

O operador `IN` testa se um valor está presente no conjunto produzido pela subconsulta.

Exemplo 9: clientes que não realizaram pedidos.

```sql
SELECT c.nome
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1 FROM pedidos AS p
    WHERE p.id_cliente = c.id_cliente
);
```

O operador `NOT EXISTS` verifica a ausência de linhas correspondentes. O resultado é **Elisa**. Trata-se de uma subconsulta *correlacionada*, pois faz referência a `c.id_cliente` da consulta externa.

6 Comparação dos assuntos

Assunto	Objetivo	Comandos
Junção horizontal	Relacionar colunas de tabelas distintas	`JOIN`, `LEFT JOIN`
Junção vertical	Combinar linhas de consultas	`UNION`, `UNION ALL`
Agregação	Calcular totais, médias e contagens	`COUNT`, `SUM`, `AVG`, `GROUP BY`, `HAVING`
Subconsultas	Usar o resultado de uma consulta em outra	`IN`, `EXISTS`, `NOT EXISTS`

7 Atividades propostas

1. Liste o nome de cada cliente e a data de seus pedidos.
2. Exiba todos os clientes, inclusive os que não compraram.
3. Compare o resultado de `UNION` e `UNION ALL` para os cadastros de 2025 e 2026.
4. Conte os produtos de cada categoria.
5. Calcule o valor total de todos os pedidos somados.
6. Apresente somente os pedidos com valor superior a R$ 100,00.
7. Liste os produtos com preço superior à média da base.
8. Identifique clientes sem pedidos utilizando `NOT EXISTS`.

A Script SQL integral para SQLite Online

O código abaixo contém a criação das tabelas, inserção dos registros, consultas demonstrativas e comandos auxiliares como `PRAGMA table_info` e `COUNT(*)`. Execute os blocos na ordem apresentada.

```sql
-- ROTEIRO PRATICO SQLITE
-- Temas: juncoes horizontais e verticais, agregacoes e subconsultas
-- Execute todo o script no https://sqliteonline.com/ ou por blocos.

DROP TABLE IF EXISTS itens_pedido;
DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS produtos;
DROP TABLE IF EXISTS clientes;
DROP TABLE IF EXISTS clientes_2025;
DROP TABLE IF EXISTS clientes_2026;

CREATE TABLE clientes (
    id_cliente INTEGER PRIMARY KEY,
    nome TEXT NOT NULL,
    cidade TEXT NOT NULL
);

CREATE TABLE produtos (
    id_produto INTEGER PRIMARY KEY,
    descricao TEXT NOT NULL,
    categoria TEXT NOT NULL,
    preco REAL NOT NULL
);

CREATE TABLE pedidos (
    id_pedido INTEGER PRIMARY KEY,
    id_cliente INTEGER NOT NULL,
    data_pedido TEXT NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);

CREATE TABLE itens_pedido (
    id_pedido INTEGER NOT NULL,
    id_produto INTEGER NOT NULL,
    quantidade INTEGER NOT NULL,
    PRIMARY KEY (id_pedido, id_produto),
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),
    FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)
);

INSERT INTO clientes VALUES
(1, 'Ana', 'Sao Luis'),
(2, 'Bruno', 'Imperatriz'),
(3, 'Carla', 'Sao Luis'),
(4, 'Diego', 'Caxias'),
(5, 'Elisa', 'Bacabal');

INSERT INTO produtos VALUES
(1, 'Sensor DHT11', 'Sensores', 25.00),
(2, 'ESP32', 'Placas', 65.00),
(3, 'Arduino Uno', 'Placas', 90.00),
(4, 'Sensor PIR', 'Sensores', 30.00),
(5, 'Modulo Rele', 'Modulos', 18.00);

INSERT INTO pedidos VALUES
(101, 1, '2026-10-01'),
(102, 2, '2026-10-02'),
(103, 1, '2026-10-05'),
```

```sql
(104, 3, '2026-10-07'),
(105, 4, '2026-10-08');

INSERT INTO itens_pedido VALUES
(101, 1, 2),
(101, 2, 1),
(102, 3, 1),
(103, 4, 3),
(104, 2, 2),
(105, 5, 4);

-- A. JUNCAO HORIZONTAL: JOIN combina colunas de tabelas relacionadas.
-- A1: clientes e respectivos pedidos (somente correspondencias)
SELECT c.nome, p.id_pedido, p.data_pedido
FROM clientes AS c
INNER JOIN pedidos AS p ON c.id_cliente = p.id_cliente
ORDER BY p.id_pedido;

-- A2: todos os clientes, inclusive os que ainda nao fizeram pedidos
SELECT c.nome, p.id_pedido
FROM clientes AS c
LEFT JOIN pedidos AS p ON c.id_cliente = p.id_cliente
ORDER BY c.nome, p.id_pedido;

-- A3: clientes, pedidos, itens e produtos
SELECT c.nome, pe.id_pedido, pr.descricao, ip.quantidade,
       ROUND(ip.quantidade * pr.preco, 2) AS subtotal
FROM clientes AS c
JOIN pedidos AS pe ON pe.id_cliente = c.id_cliente
JOIN itens_pedido AS ip ON ip.id_pedido = pe.id_pedido
JOIN produtos AS pr ON pr.id_produto = ip.id_produto
ORDER BY pe.id_pedido;

-- B. JUNCAO VERTICAL: UNION empilha linhas de consultas compativeis.
CREATE TABLE clientes_2025 (nome TEXT, cidade TEXT);
CREATE TABLE clientes_2026 (nome TEXT, cidade TEXT);
INSERT INTO clientes_2025 VALUES ('Ana', 'Sao Luis'), ('Bruno', 'Imperatriz');
INSERT INTO clientes_2026 VALUES ('Ana', 'Sao Luis'), ('Carla', 'Sao Luis');

-- B1: UNION elimina linhas duplicadas
SELECT nome, cidade FROM clientes_2025
UNION
SELECT nome, cidade FROM clientes_2026;

-- B2: UNION ALL preserva duplicatas
SELECT nome, cidade FROM clientes_2025
UNION ALL
SELECT nome, cidade FROM clientes_2026;

-- C. FUNCOES DE AGREGACAO
-- C1: numero de clientes
SELECT COUNT(*) AS total_clientes FROM clientes;

-- C2: quantidade de produtos por categoria
SELECT categoria, COUNT(*) AS quantidade_produtos
FROM produtos
GROUP BY categoria
ORDER BY categoria;

```

```sql
-- C3: preco medio, minimo e maximo de cada categoria
SELECT categoria,
       ROUND(AVG(preco), 2) AS preco_medio,
       MIN(preco) AS menor_preco,
       MAX(preco) AS maior_preco
FROM produtos
GROUP BY categoria;

-- C4: total vendido por pedido
SELECT ip.id_pedido,
       ROUND(SUM(ip.quantidade * pr.preco), 2) AS valor_total
FROM itens_pedido AS ip
JOIN produtos AS pr ON pr.id_produto = ip.id_produto
GROUP BY ip.id_pedido
ORDER BY ip.id_pedido;

-- C5: pedidos com valor total superior a R$ 100
SELECT ip.id_pedido,
       ROUND(SUM(ip.quantidade * pr.preco), 2) AS valor_total
FROM itens_pedido AS ip
JOIN produtos AS pr ON pr.id_produto = ip.id_produto
GROUP BY ip.id_pedido
HAVING SUM(ip.quantidade * pr.preco) > 100;

-- D. SUBCONSULTAS
-- D1: produtos com preco superior a media geral
SELECT descricao, preco
FROM produtos
WHERE preco > (SELECT AVG(preco) FROM produtos)
ORDER BY preco DESC;

-- D2: clientes que fizeram pelo menos um pedido (IN)
SELECT nome
FROM clientes
WHERE id_cliente IN (SELECT id_cliente FROM pedidos)
ORDER BY nome;

-- D3: clientes sem pedidos (NOT EXISTS, subconsulta correlacionada)
SELECT c.nome
FROM clientes AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM pedidos AS p
    WHERE p.id_cliente = c.id_cliente
);

-- D4: clientes com mais de um pedido
SELECT c.nome, COUNT(p.id_pedido) AS quantidade_pedidos
FROM clientes AS c
JOIN pedidos AS p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) > 1;

-- E. CONSULTAS UTEIS NO SQLITE ONLINE
-- Listar as tabelas do banco
SELECT name FROM sqlite_master WHERE type = 'table' AND name NOT LIKE 'sqlite_%' ORDER
    BY name;
-- Estrutura de uma tabela
PRAGMA table_info(clientes);
```

```sql
-- Numero de registros
SELECT COUNT(*) AS total FROM pedidos;
```